



Centrifuge Bug Bounty Audit Report

Prepared by [Burra Security](#)

Version 1.0

Researchers

Alex Filippov

August 10, 2026

Contents

1	About Burra Security	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	4
7.1	Medium Risk	4
7.1.1	Missing deduplication in MultiAdapter quorum=1 fast path	4
7.1.2	Pending deposit requests reduce the available balance in PoolEscrow and block withdrawals	5
7.2	Low Risk	7
7.2.1	Unchecked controller lets attackers queue another user's next deposit or redeem	7
7.2.2	PROCESS_FAIL_MESSAGE_GAS may be insufficient on Monad	7
7.2.3	Missing gas cap on isPoolEscrow() staticcall enables gas grieving on compliance operations	8
7.2.4	Incorrect msgValue() handling during batching can leave Safe ETH stuck in Hub	9
7.3	Informational	11
7.3.1	Unused MathLib import in PoolId.sol	11
7.3.2	QueueManager inherits Auth without auth-gated functionality	11
7.3.3	BalanceSheet does not explicitly initialize the batched multicall gateway	11
7.3.4	V2CleaningsSpell does not clean legacy vault wards	11
7.3.5	Cancel-only deposit fulfilment can invalidate a stale maxDeposit	12
7.3.6	Sync vault Deposit events appear to invert sender and owner	12
7.3.7	Failed messages cannot be invalidated	12
7.3.8	Holding account distinctness check is incomplete	13

1 About Burra Security

Burra Security offers security auditing and advisory services with a special focus on cross-chain and interoperability protocols and their integrations. Learn about us at burrasec.com.

2 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Centrifuge is an open protocol for onchain asset tokenization built on a hub-and-spoke architecture. Each pool designates a single hub chain where pool accounting, share class management, and on-chain NAV computation are coordinated through Hub, Accounting, Holdings, and ShareClassManager; spoke chains host the user-facing vault and escrow infrastructure.

On spoke chains, pools expose per-asset balance sheets, share tokens, and vault registries supporting both synchronous deposits and asynchronous request-based deposit/redemption flows. Transfer restrictions are configurable per share class via pluggable hooks, FullRestrictions, FreezeOnly, RedemptionRestrictions, or FreelyTransferable, enabling compliance-ready tokenization of permissioned funds alongside open DeFi products. Asset pricing is modular, backed by OracleValuation and IdentityValuation implementations.

Vaults implement ERC-7540 (async request flows) and ERC-7575, integrating natively into DeFi composability layers. Cross-chain messaging is handled by a MultiAdapter gateway routing over Axelar, LayerZero, Wormhole, and Chainlink adapters simultaneously, providing fault-tolerant delivery across Cancun-compatible EVM chains. On- and off-ramp flows are served by OnOfframpManager with Circle and Vault decoder support, while MerkleProofManager enables trust-minimized spoke-side claim verification.

5 Audit Scope

The audit scope consisted of all deployed contracts in the [Centrifuge protocol v3.1.0 release](#) at commit [6b9d36eabee48728486f377ea2766a5cd233c555](#), as defined by the [Centrifuge bug bounty program on Cantina](#).

6 Executive Summary

Over the course of 40 days, the Burra Security team conducted an audit on the [Centrifuge Bug Bounty](#) smart contracts provided by [Centrifuge](#). In this period, a total of 14 issues were found.

Summary

Project Name	Centrifuge Bug Bounty
Repository	v3.1.0
Commit	6b9d36eabee4...
Audit Timeline	Mar 27th - May 21st
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	4
Informational	8
Gas Optimizations	0
Total Issues	14

Summary of Findings

[M-1] Missing deduplication in MultiAdapter quorum=1 fast path	Resolved
[M-2] Pending deposit requests reduce the available balance in PoolEscrow and block withdrawals	Resolved
[L-1] Unchecked controller lets attackers queue another user's next deposit or redeem	Acknowledged
[L-2] PROCESS_FAIL_MESSAGE_GAS may be insufficient on Monad	Resolved
[L-3] Missing gas cap on isPoolEscrow() staticcall enables gas griefing on compliance operations	Acknowledged
[L-4] Incorrect msgValue() handling during batching can leave Safe ETH stuck in Hub	Resolved
[I-1] Unused MathLib import in PoolId.sol	Resolved
[I-2] QueueManager inherits Auth without auth-gated functionality	Resolved
[I-3] BalanceSheet does not explicitly initialize the batched multicall gateway	Resolved
[I-4] V2CleaningsSpell does not clean legacy vault wards	Resolved
[I-5] Cancel-only deposit fulfilment can invalidate a stale maxDeposit	Acknowledged
[I-6] Sync vault Deposit events appear to invert sender and owner	Resolved
[I-7] Failed messages cannot be invalidated	Resolved
[I-8] Holding account distinctness check is incomplete	Acknowledged

7 Findings

7.1 Medium Risk

7.1.1 Missing deduplication in MultiAdapter quorum=1 fast path

Target: [MultiAdapter.sol](#)

Severity:

- Impact: High
- Likelihood: Low

Description: A cross-chain message can be executed twice when the adapter configuration for a pool changes from multi-adapter to single-adapter (Axelar).

`AxelarAdapter.execute()` is publicly callable. It validates via the Axelar gateway's pull-based `validateContractCall()`, which atomically consumes the `commandId`, but has no `msg.sender` restriction — anyone can trigger it. When another adapter delivers a message first and the Axelar `commandId` goes unconsumed, that `commandId` remains valid indefinitely in the Axelar gateway. If a later `SetPoolAdapters` reconfigures the pool to Axelar-only (`quorum=1`), anyone can call `AxelarAdapter.execute()` with the stale `commandId`, and the message executes a second time via the `quorum=1` fast path, which has no deduplication:

```
if (adapter.quorum == 1) {
    gateway.handle(centrifugeId, payload);
    return;
}
```

Neither `MultiAdapter` nor `Gateway` track executed message hashes.

Attack Scenario:

Setup: Spoke chain, pool P with 2 adapters (LZ, Axelar), `threshold=1`.

1. Hub sends `ExecuteTransferShares` message M to the spoke. Both adapters carry a copy.
2. LZ delivers M first → threshold met → M executes → **shares minted**. Axelar's `commandId` for M remains unconsumed.
3. Pool manager reconfigures pool P to Axelar-only (`quorum=1`) via `SetPoolAdapters`.
4. Attacker calls `AxelarAdapter.execute(staleCommandId, sourceAxelarId, sourceAddress, payloadM)`. The Axelar gateway validates and consumes the `commandId`. `MultiAdapter.handle()` sees `quorum=1` → fast path → M executes again → **shares minted a second time**.

The attacker obtains the payload from the `HandlePayload` event emitted during step 2.

Impact: `Spoke.executeTransferShares()` unconditionally mints and transfers shares with no replay guard:

```
function executeTransferShares(PoolId poolId, ShareClassId scId, bytes32 receiver, uint128 amount) public
↳ auth {
    shareToken_.mint(address(this), amount);
    shareToken_.transfer(receiver.toAddress(), amount);
}
```

Double execution inflates the share supply and grants the receiver unearned shares. Some message types are naturally protected (e.g. `UpdateHoldingAmount` and `UpdateShares` revert on nonce mismatch in `Holdings.setSnapshot`; `FulfilledDepositRequest`/`FulfilledRedeemRequest` revert when pending counters are zero), but `ExecuteTransferShares` is not.

Recommended Mitigation: Consider taking the `quorum=1` fast path only when `inbound[centrifugeId][payloadHash].sessionId == 0`, which means the payload has no prior multi-adapter vote state. If prior state exists, route the message through the session-transition path instead. That transition logic should be fixed separately because the current reset behaviour has known replay issues.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/165>.

BurraSec: Fix looks good. MultiAdapter was significantly refactored in Centrifuge v3.3, and the quorum-1 fast path no longer exists.

7.1.2 Pending deposit requests reduce the available balance in PoolEscrow and block withdrawals

Target: [PoolEscrow.sol](#)

Severity:

- Impact: Medium
- Likelihood: Medium

Description: PoolEscrow keeps a single Holding{total, reserved} per (shareClassId, asset, tokenId), and withdraw() only allows withdrawing from total - reserved:

```
function withdraw(ShareClassId scId, address asset, uint256 tokenId, address receiver, uint128 value)
    external
    auth
{
    Holding storage holding_ = holding[scId][asset][tokenId];
    require(holding_.total >= holding_.reserved, InsufficientBalance(asset, tokenId, value, 0));

    uint128 balance = holding_.total - holding_.reserved;
    require(balance >= value, InsufficientBalance(asset, tokenId, value, balance));

    holding_.total -= value;
    emit Withdraw(asset, tokenId, poolId, scId, receiver, value);
}
```

Pending deposit requests increase holding.reserved, but do not increase holding.total at the same time. AsyncRequestManager.requestDeposit() immediately reserves the asset with REASON_DEPOSIT, while AsyncVault.requestDeposit() transfers the tokens into the pool escrow. However, holding.total is only increased later in approvedDeposits() via balanceSheet.noteDeposit() -> escrow.deposit(). As a result, pending deposits reduce the tuple's available balance even though the funds are already sitting in PoolEscrow.

The issue arises when async deposit flow interacts with redeem flow. revokedShares() first reserves the payout asset with REASON_REDEEM and defers the escrow-side withdrawal by calling noteWithdraw(...) without updating escrow. Later, when the user claims, _processRedeem() first unreserves the user's redeem reservation and only then calls balanceSheet.withdraw(..., WithdrawMode.EscrowAndTransfer), which reaches PoolEscrow.withdraw(). Because all reservation reasons share the same holding.reserved counter, unrelated pending deposit reservations can make the withdrawal fail.

Attack Scenario:

1. For some (shareClassId, asset, tokenId), holding.total = 1000 and holding.reserved = 990. Out of the 990 reserved, 900 belongs to the victim's approved redeem. The remaining 90 comes from other reservations.
2. The victim tries to claim 900.
3. Before the claim executes, the attacker submits requestDeposit(11) for the same tuple.
4. This increases holding.reserved to 1001, while holding.total stays 1000.
5. During the victim's claim, _processRedeem() unreserves the victim's 900, so holding.reserved becomes 101.
6. PoolEscrow.withdraw() now sees available balance 1000 - 101 = 899.
7. Since 899 < 900, the victim's claim reverts.

The DoS lasts until the hub sends the `approvedDeposits` callback for the attacker's deposit.

Impact: Temporary, selective DoS of all withdrawal paths (`_processRedeem`, `OnOfframpManager.withdraw`) for a chosen (`shareClassId`, `asset`, `tokenId`).

Recommended Mitigation: Consider refactoring `PoolEscrow` reservation accounting to distinguish reservations against already-accounted pool holdings from pre-accounting custody locks.

`holding.reserved` should only track amounts that are backed by `holding.total` and therefore must reduce `availableBalanceOf()` / `block withdraw()`. Pending async deposit assets, which are held in escrow before being reflected in `holding.total`, should be tracked separately so they cannot reduce unrelated withdrawal liquidity.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/250>.

BurraSec: Fix looks good. `requestDeposit()` now increases `holding.total` through `noteDeposit()` and reserves the same amount, while `approvedDeposits()` only releases that reservation. Pending async deposits therefore no longer reduce the escrow's available balance and cannot cause the reported withdrawal DoS.

7.2 Low Risk

7.2.1 Unchecked controller lets attackers queue another user's next deposit or redeem

Target: [AsyncVault.sol](#), [BaseVaults.sol](#), [BatchRequestManager.sol](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: `AsyncVault.requestDeposit/assets, controller, owner` and `BaseAsyncRedeemVault.requestRedeem/shares, controller, owner` check that `msg.sender` is owner (or its operator), but never validate controller. controller is forwarded to the Hub and used as the investor key in `depositRequest[...][investor]` / `redeemRequest[...][investor]`. Any address holding 1 wei of the asset (or 1 share) can call `requestDeposit(1, victim, attacker)` or `requestRedeem(1, victim, attacker)` and create a pending request under the victim's key.

On the hub, `BatchRequestManager._updatePending` sets `userOrder.pending = 1` and `userOrder.lastUpdate = E`, where `E` is the current deposit or redeem epoch. The manager then approves epoch `E`, which advances `epochId` so the next epoch becomes `E + 1`.

When the victim submits their own request, the hub evaluates `_canMutatePending`:

```
return currentEpoch <= 1 || userOrder.pending == 0 || userOrder.lastUpdate >= currentEpoch;
```

The victim's pending is now 1 (from the attacker) and `lastUpdate` is `E`, which is less than `E + 1`, so the check fails. `_updateQueued` stores the victim's amount in `queuedRedeemRequest[...][victim].amount` instead of increasing the global `pendingRedeem[...]` counter. The manager only approves from the global counter, so the victim's real request is not included in epoch `E + 1`'s approval.

Impact: An attacker can, in some cases, delay a user's deposit or redeem by at least one epoch. Because the victim (or someone on their behalf) must call `notifyDeposit` / `notifyRedeem` to flush the queued state into pending, this is also a grieving vector: the attacker can donate from a cheap spoke chain (potentially subsidized by the pool), while the victim is forced to pay for an additional `notify*` call on mainnet or another expensive chain.

Recommended Mitigation: Consider validating controller in `AsyncVault.requestDeposit` and `BaseAsyncRedeemVault.requestRedeem` using `BaseVault._validateController`.

Client: Deferred to a later upgrade, as it is hard to change now and the severity is low.

BurraSec: Acknowledged.

7.2.2 PROCESS_FAIL_MESSAGE_GAS may be insufficient on Monad

Target: [PROCESS_FAIL_MESSAGE_GAS = 35_000](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: `Gateway._safeProcess` reserves `PROCESS_FAIL_MESSAGE_GAS = 35_000` for the failure branch after calling the processor.

```
function _safeProcess(uint16 centrifugeId, bytes memory message, bytes32 messageHash, uint128 gasLimit)
↳ internal {
    (bool success, bytes memory err) = address(processor)
        .excessivelySafeCall(
            gasLimit - PROCESS_FAIL_MESSAGE_GAS,
            0,
            ERR_MAX_LENGTH,
            abi.encodeWithSelector(IMessageHandler.handle.selector, centrifugeId, message)
```

```

    );

    if (success) {
        emit ExecuteMessage(centrifugeId, messageHash);
    } else {
        failedMessages[centrifugeId][messageHash]++;
        emit FailMessage(centrifugeId, messageHash, err);
    }
}

```

On Monad, cold access costs are **higher** than Ethereum:

- Cold SLOAD: 2100 (Ethereum) -> 8100 (Monad)
- Cold account access (CALL target): 2600 (Ethereum) -> 10100 (Monad)

The failure tail in `_safeProcess` relies on these cold accesses (read processor, call processor, read/update `failedMessages`). That repricing adds +19,500 gas versus Ethereum assumptions for the same tail in the worst case.

The current **benchmark** already measures failure-path consumption at 34,711 gas, leaving only 289 gas under the 35,000 reserve. With Monad repricing, expected consumption is above the reserve, so the caller-side failure tail can run out of gas.

When this happens:

1. The entire `Gateway.handle(...)` call reverts, so `failedMessages[centrifugeId][messageHash]` is never incremented.
2. `Gateway.retry(...)` cannot be used for that message because it requires a recorded failed count.
3. LZ, Axelar, and Chainlink allow redelivery on the destination chain with an increased gas limit (no source-chain action required).
4. For Wormhole-routed batches, retrying is not straightforward. Recovery requires calling `IWormholeRelayer.resendToEvm(deliveryVaaKey, targetChain, newReceiverValue, newGasLimit, newDeliveryProviderAddress)` on the source chain, where `deliveryVaaKey = {emitterChainId, emitterAddress, sequence}` and `newGasLimit >= original`.

Impact:

- Cross-chain message processing can stall on Monad for affected batches. Recovery may require manual intervention. Given that a malicious pool manager could, in some cases, include honest messages in their batch (see issue *Incorrect `msgValue()` handling during batching can leave Safe ETH stuck in Hub*), this vulnerability could be used to temporarily DoS messages to Monad.
- The same general issue also affects `BASE_COST`, adapter `RECEIVE_COST`, and the gas estimates in `GasService`.

Recommended Mitigation: The `BASE_COST` / `GasService` part can be addressed by using a separate Monad-specific `GasService` deployment.

However, `RECEIVE_COST` and `PROCESS_FAIL_MESSAGE_GAS` are still fixed in code. It may make sense to refactor this part as well, for example by moving these values into `GasService` or otherwise making them configurable per deployment.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/188>.

BurraSec: Fix looks good. The `PROCESS_FAIL_MESSAGE_GAS` constant has been moved into `GasService`, which now applies a separate failure gas reserve for Monad.

7.2.3 Missing gas cap on `isPoolEscrow()` staticcall enables gas griefing on compliance operations

Target: [BaseTransferHook.sol#L128](#)

Severity:

- Impact: Low
- Likelihood: Medium

Description: In multi-pool mode (`poolEscrow == address(0)`), `isPoolEscrow()` performs a `staticcall` to the target address's `poolId()` without a gas limit:

```
(bool success, bytes memory data) = addr.staticcall(abi.encodeWithSignature("poolId()"));
```

The `staticcall` forwards 63/64 of available gas (EIP-150) to an arbitrary contract. A malicious contract with a gas-draining `poolId()` can consume nearly all forwarded gas. `isPoolEscrow()` is called from `freeze()` and `updateMember()` on the user parameter.

The malicious `poolId()` drains 63/64 of all gas forwarded to the `staticcall` (EIP-150), leaving only 1/64 for the remaining execution (`hookDataOf()` + `setHookData()` + `event`, ~25K gas needed). This forces `freeze()` and `updateMember()` to require ~1.56M gas instead of ~46K — a ~33x increase. The ratio is 33x rather than 64x because the drain only amplifies the post-`staticcall` portion; pre-`staticcall` overhead (~15K gas) is unaffected.

The default cross-chain gas budget of 112,670 (`GasService`) cannot cover the inflated cost, so `UpdateRestriction` messages fail and are stored in `failedMessages`.

Attack Scenario:

1. Attacker deploys a contract whose `poolId()` function burns all forwarded gas.
2. Attacker holds share tokens at this address.
3. Pool manager sends a cross-chain `UpdateRestriction` (freeze) message targeting the malicious contract with the default `extraGasLimit = 0`.
4. On the spoke chain, `Gateway._safeProcess` dispatches the message with 112,670 gas.
5. The malicious `poolId()` drains 63/64 of the gas available at `freeze()` → `isPoolEscrow()`.
6. `freeze()` reverts with OOG. The message is stored in `failedMessages` and a `FailMessage` event is emitted.
7. The attacker monitors for the `FailMessage` event and transfers tokens to a clean address before the compliance manager retries or freezes directly.

Impact: An attacker can force `freeze()` and `updateMember()` to consume ~33x more gas than normal (~1.56M vs ~46K). This causes cross-chain messages to fail with the default gas budget, since 112,670 gas is far below the ~1.56M required. The manager can recover via `Gateway.retry()`, direct spoke-chain calls, or resending with a higher `extraGasLimit`. Single-pool deployments (`poolEscrow != address(0)`) are unaffected.

Recommended Mitigation: Cap the gas forwarded to the `staticcall`.

Client: Acknowledged (low severity).

BurraSec: Acknowledged.

7.2.4 Incorrect `msgValue()` handling during batching can leave Safe ETH stuck in Hub

Target: [BatchedMulticall.sol](#)

Severity:

- Impact: Medium
- Likelihood: Low

Description: The Hub contract inherits `BatchedMulticall`, where `msgValue()` returns 0 whenever batching is active:

```
/// @dev Only the call to multicall should pass the msg.value, which is then passed
/// in `gateway.withBatch`. No inner calls should pass any msg.value.
function msgValue() internal view returns (uint256 value) {
```

```
    return _sender != address(0) ? 0 : msg.value;
}
```

This behavior depends only on `_sender != address(0)`. Unlike `msgSender()`, it does not verify that the current call came from the intended Gateway callback path.

`Hub.multicall()` keeps `_sender` set for the entire batch. During that window, `Hub.updateHoldingValue()` may invoke an external snapshot hook through `holdings.callOnSyncSnapshot()`. Because this is a normal external call, the hook can execute arbitrary code while batching is still active.

```
function updateHoldingValue(PoolId poolId, ShareClassId scId, AssetId assetId) public payable {
    _isManager(poolId);

    (bool isPositive, uint128 diff) = holdings.update(poolId, scId, assetId);
    _updateAccountingValue(poolId, scId, assetId, isPositive, diff);

    holdings.callOnSyncSnapshot(poolId, scId, assetId.centrifugeId());
}
```

As a result, any direct reentrant call into Hub during `onSync()` is treated as if it carried no ETH, even if real ETH was attached.

Attack Scenario:

1. Manager A is a malicious pool manager
2. Manager B is a Safe and has already signed a valid regular Hub operation with excessive `msg.value` (e.g. 10 ETH), expecting the normal refund flow.
3. Manager A calls: `Hub.multicall([setSnapshotHook(maliciousHook), updateHoldingValue(...)])`
4. `_sender = ManagerA` is cached for the duration of the batch.
5. `updateHoldingValue()` triggers `hook.onSync(...)`.
6. During `onSync()`, the malicious hook executes Manager B's already-signed Safe transaction.
7. That Safe transaction calls a payable Hub function, such as `notifyPool(...)`, with nonzero ETH.
8. Inside Hub, `msgValue()` returns 0 because batching is active.
9. The operation is processed as if no ETH was supplied, while the actual ETH sent with the Safe transaction is not forwarded through the intended path and will remain stuck in Hub.

Impact: A malicious pool manager can use a snapshot hook to execute a pre-signed Safe operation within a batching context. The Safe operation itself will still succeed, but all ETH attached to it will be stuck in Hub instead of following the intended downstream refund flow. The stuck ETH could be recovered later via `recoverTokens`, but that would require manual action from the ProtocolGuardian.

Recommended Mitigation: Consider updating `msgValue()` to mirror `msgSender()` and return 0 only when `_sender != address(0) && msg.sender == address(gateway)`.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/214>.

BurraSec: Fix looks good. `msgValue()` now mirrors `msgSender()`: the value is only zeroed for calls coming through the gateway callback path.

7.3 Informational

7.3.1 Unused MathLib import in PoolId.sol

Target: [PoolId.sol#L6-L6](#)

Severity:

- Informational

Description: [PoolId.sol#L6-L6](#) imports MathLib and attaches it to uint256, but no function in the file operates on uint256.

Recommended Mitigation: Consider removing unused import.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/256>.

BurraSec: Fix looks good.

7.3.2 QueueManager inherits Auth without auth-gated functionality

Target: [QueueManager.sol#L20-L30](#)

Severity:

- Informational

Description: [QueueManager.sol#L20-L30](#) inherits Auth and initializes an initial ward, but no function in the contract uses the auth modifier. Its privileged configuration path is instead gated by contractUpdater, while sync is permissionless and syncCallback is protected by the gateway callback lock. This leaves unnecessary authorization surface.

Recommended Mitigation: Consider removing the Auth inheritance and deployer ward initialization unless future ward-gated functionality is intended.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/256>.

BurraSec: Fix looks good.

7.3.3 BalanceSheet does not explicitly initialize the batched multicall gateway

Target: [BalanceSheet.sol#L51-L51](#)

Severity:

- Informational

Description: [BalanceSheet.sol#L51-L51](#) passes the inherited gateway state variable to BatchedMulticall during construction, before that variable is configured. The deployment script later files the real gateway, but the constructor dependency is implicit and differs from the other BatchedMulticall integrations.

Recommended Mitigation: Consider adding a gateway_ constructor parameter to BalanceSheet and passing it to BatchedMulticall(gateway_) to make the dependency explicit.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/255>.

BurraSec: Fix looks good. BalanceSheet was merged into Spoke, which takes IGateway gateway_ as a constructor parameter and passes it to BatchedMulticall(gateway_).

7.3.4 V2CleaningsSpell does not clean legacy vault wards

Target: [V2CleaningsSpell.sol#L76-L94](#)

Severity:

- Informational

Description: [V2CleaningsSpell.sol#L76-L94](#) removes `ROOT_V2` from the migrated JAAA and JTRSY share tokens, but it only targets that root ward and does not deny legacy V2 vault wards that may also remain on those share tokens. For example, on Base, `BASE_V2_JAAA_VAULT` (`0xB4C8540657d67D4846cAe68EcF2C706c80DC3c9`) remains a ward of JAAA and `BASE_V2_JTRSY_VAULT` (`0xF9a6768034280745d7F303D3d8B7f2bF3Cc079eF`) remains a ward of JTRSY.

Recommended Mitigation: Consider extending the cleanup spell to deny known legacy V2 vault addresses from the affected share tokens.

Client: Fixed in <https://github.com/centrifuge/protocol/pull/818>.

BurraSec: Fix looks good. `_disableRootV2FromShareTokensV2` now denies the legacy V2 vaults on each share token.

7.3.5 Cancel-only deposit fulfilment can invalidate a stale `maxDeposit`

Target: [AsyncRequestManager.sol#L299-L301](#)

Severity:

- Informational

Description: [AsyncRequestManager.sol#L299-L301](#) recalculates `depositPrice` even for cancellation-only deposit fulfilments where no new assets or shares are fulfilled. Because the new price is derived from `_maxDeposit`, which is already rounded down, this extra conversion can lower the user's asset-denominated `maxDeposit` by a small amount. A transaction that previously read `vault.maxDeposit(user)` and then calls `deposit` with that stale value can therefore revert, although funds are not lost and the user can retry with the updated amount.

Recommended Mitigation: Consider skipping the `depositPrice` recalculation when `fulfilledAssets` and `fulfilledShares` are both zero.

Client: Acknowledged.

BurraSec: Acknowledged.

7.3.6 Sync vault `Deposit` events appear to invert sender and owner

Target: [BaseVaults.sol#L376-L399](#)

Severity:

- Informational

Description: [BaseVaults.sol#L376-L399](#) emits `Deposit(receiver, msg.sender, assets, shares)` from the sync deposit and mint flows. However, [IERC7575.sol#L7-L7](#) defines the event as `Deposit(sender, owner, assets, shares)`, so the emitted indexed fields appear to swap the depositor and receiver.

Recommended Mitigation: Consider emitting `Deposit(msg.sender, receiver, assets, shares)`.

Client: Fixed in <https://github.com/centrifuge/protocol-internal/pull/256>.

BurraSec: Fix looks good.

7.3.7 Failed messages cannot be invalidated

Target: [Gateway.sol#L130-L144](#)

Severity:

- Informational

Description: [Gateway.sol#L130-L144](#) records failed messages by hash and allows anyone to retry them later, but the contract has no admin path to invalidate a failed message. A message sent by a previously-authorized manager can therefore remain retryable even after the manager is removed or the intended configuration changes, as long as the message originally reached the destination and failed during processing.

Recommended Mitigation: Consider adding an auth-gated invalidation path for failed message hashes.

Client: Addressed in <https://github.com/centrifuge/protocol-internal/pull/227>.

BurraSec: Fix looks good. The newly added `clearFailedMessage` allows a manager to remove failed messages.

7.3.8 Holding account distinctness check is incomplete

Target: [Hub.sol#L339-L342](#)

Severity:

- Informational

Description: [Hub.sol#L339-L342](#) reverts with `InvalidAccountCombination` when the asset account equals the equity, gain, or loss account. This appears intended to ensure that the holding uses distinct accounting roles, but the check does not compare the equity, gain, and loss accounts with each other. As a result, duplicate non-asset role accounts can still pass this validation.

Recommended Mitigation: Consider checking all pairwise account combinations.

Client: Addressed in <https://github.com/centrifuge/protocol-internal/pull/248>.

BurraSec: Acknowledged. The distinctness check was dropped in the accounting redesign, so reusing an account across roles is now intentional.